# PEGAI

# MINI PROJECT

PES1UG24AM293 PES1UG24AM240 PES1UG24AM251 PES1UG24AM252

**1. Problem Statement**

Long technical, scientific, and policy documents often contain thousands of tokens, making them difficult and expensive for Large Language Models (LLMs) to process within a fixed context-window budget. Our project aims to develop a system that can **summarize long documents efficiently while reducing the number of input tokens** through prompt compression.

We will use **30–50 documents from the GovReport or arXiv summarization datasets** and compare different document chunking, summarization, and prompt-compression techniques. The main objective is to study the trade-off between **token savings and summary quality**.

**Research Question:**
*How much can we compress a long-document prompt while maintaining good summary quality, coverage, and factual faithfulness?*

**2. Why Does It Matter?**

Long-document summarization is important for government reports, research papers, technical documents, legal documents, and policy reports. Sending an entire document to an LLM can cause context-window limitations, high token consumption and cost, slow processing, loss of important information, and hallucinations or factual errors.

Prompt compression techniques such as LLMLingua attempt to remove redundant tokens while preserving important information. Therefore, our project investigates whether we can save tokens without significantly reducing the quality of the generated summary.

**3. Approaches We Can Use**

Chunking Strategies: Token-based chunking divides the document into fixed-size token chunks and provides a simple baseline. Recursive chunking splits according to natural boundaries such as sections, paragraphs, and sentences, helping preserve document structure. Semantic chunking uses semantic similarity or embeddings to group related content together and may produce more meaningful chunks, although it is more computationally expensive.

Summarization Chains: Stuff combines available text and sends it to the LLM in one prompt; it is simple but unsuitable for extremely long documents. Map-Reduce summarizes each chunk separately and then combines the intermediate summaries into a final summary, making it suitable for long documents. Refine processes chunks sequentially and continuously refines the existing summary as new information is encountered.

**4. Prompt Compression**

We can apply LLMLingua before sending the chunks or prompts to the LLM. We can compare: Baseline – no compression; Low – 2× compression; Medium – 4× compression; High – 6× compression. For example, a 10,000-token prompt compressed by 4× would contain approximately 2,500 tokens.